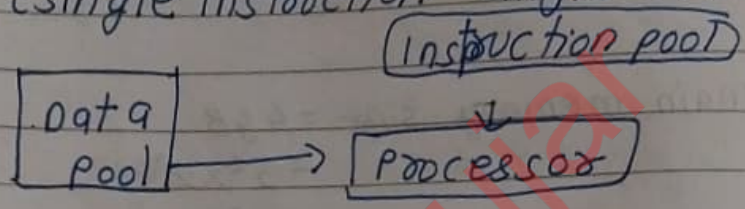
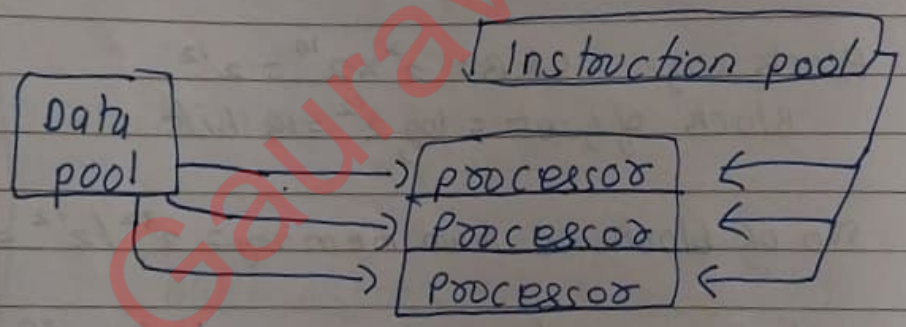
Flynn's classification

- stream
- Instruction stream
- Data stream

SISD (single Instruction Single data) system



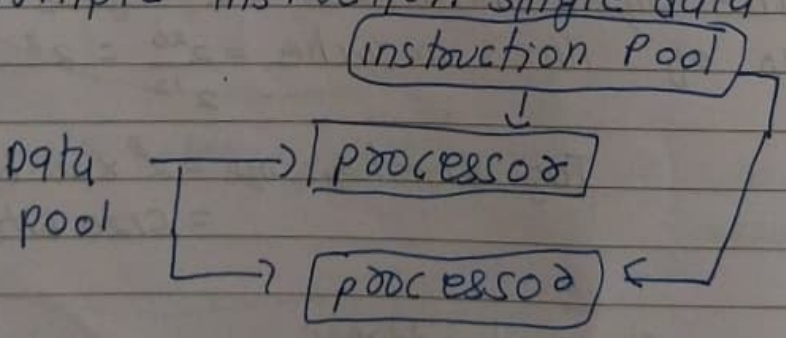
SIMD (single instruction multiple data) system



multiple Instruction single data system

mov, 50
ADD AL, BL
mov BL, AL

mov 60
ADD AL, BL
mov BL, AL

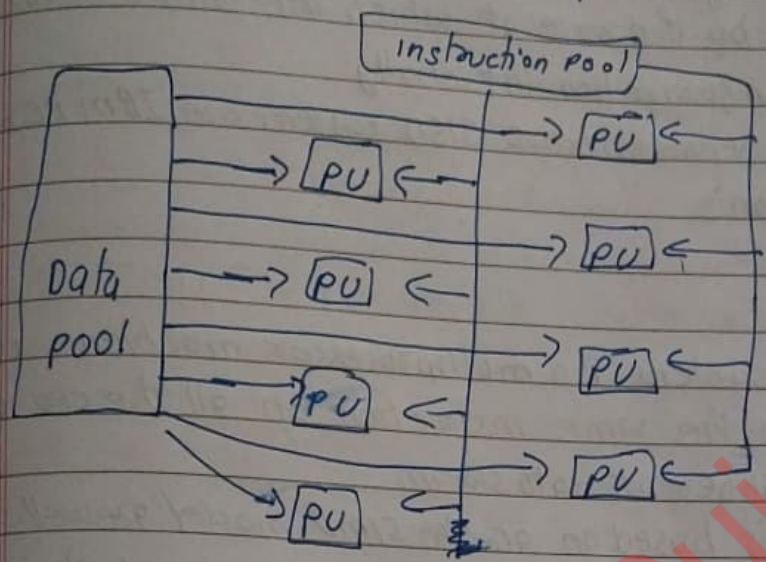


mov AL, 50
ADD AL, BL

MOV AL, 8
MUL AL, BL

Gaurav Pujari

multiple instruction multiple data system



mov	AL, 60	mov	AL, 50
ADD	AL, BL	mult	AL, BL
mov	DL, AL	mov	DL, AL
mov	CX, DX	mov	CL, DL

i An SISD computing system is a uniprocessor machience which is capable of executing a single instruction and operating on a single data stream

ii In SISD machine instruction are processed in sequential manner and computer's adopting this model are popularly called sequential computer's

iii Most conventional computer have SISD architecture

iv All the instruction and data to be processed have to be stored in primary memory

Gaurav Pujar

v The speed of the processing element in SISD model is limited by the rate at which the computer can transfer information internally

vi Dominant representative SISD system are IBM PC, work station's

b] SIMD

i An SIMD system is a multiprocessor machiene capable of executing the same instruction on all the CPU but operating on dibferent data stream

ii machiene's based on an ~~SIM~~ SIMD model are well suited to scientific computing since they involve lot's of vector and matrix operation

iii So that the information can be passed to all the processing element, ~~or~~ organise data elements of vector can be divided into multiple sets (N - sets for NPE system) and each PE processing element can process one data set

iv Dominent Representative SIMD system is ~~Cray~~ Cray's vector processing machiene

c] MISD

i An MISD ~~m~~ computing system is a multiprocessor machiene capable of executing different instruction on different processing element but all of them operating on some dataset

ii machiene built using MISD model is not useful is most of application model, a few machiene are build but none of them are available comercially

Gaurav Pujar

d] MIMD

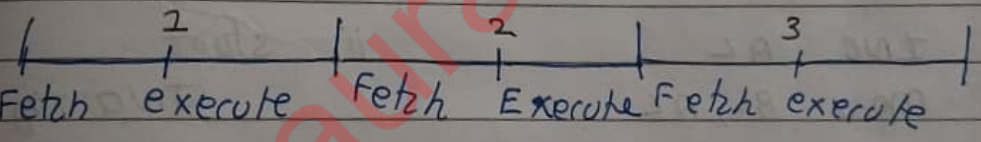
An MIMD system is a multiprocessor machience which is capable of executing multiple instruction on multiple data set ~~each proc~~

ii each processing element in the MIMD model has seperate instruction and data stream Therefore machiene build using this model are capable to any kind of application

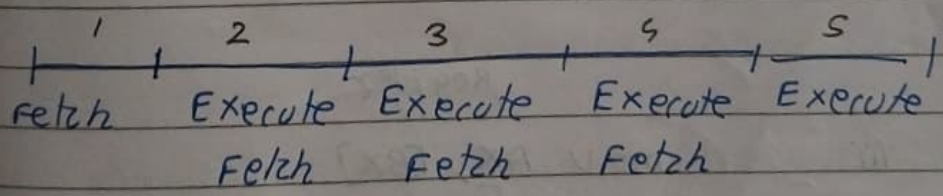
iii unlike SIMD & MISD machiene processing element in MIMD machience work achronously

Instruction pipeline :-

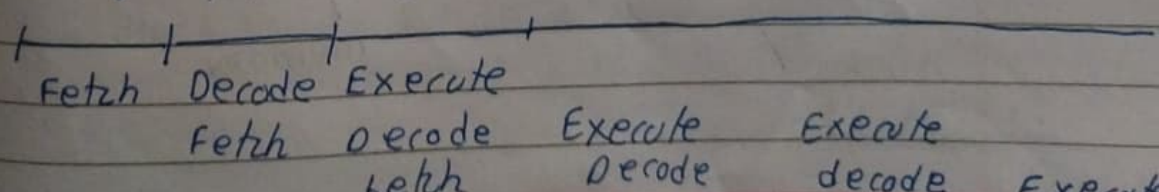
1] 8085 execution



2 8086 execution (2stage pipelining)



3 3 stage pipelining



Gaurav Pujari

4) 4 stage pipelining

Fetch	Decode	Execute	store	store	store	store
	Fetch	decode	execute	execute	execute	~~fetch~~
		fetch	decode	decode	~~decode~~	
			fetch	~~fetch~~		

pipeline hazard

1) Data hazard / Data dependancy hazard
2) control hazard / code hazard
3) structure hazard.

i
```
INC AL
mov BL, AL
```

AL = ~~8~~ ⇒ 6

mov BL, AL

S ↳ But value is 6

ii
```
start
    JNP Tag   ← ①
    DEC AL    ← ②
    :
Tag
    DEC DL1   -- it should be 2
```

iii

mov AX, [BX]

Register

mov BX, [2000H]

memory

Gaurav Pujar

Q3] what is paging mechanism?

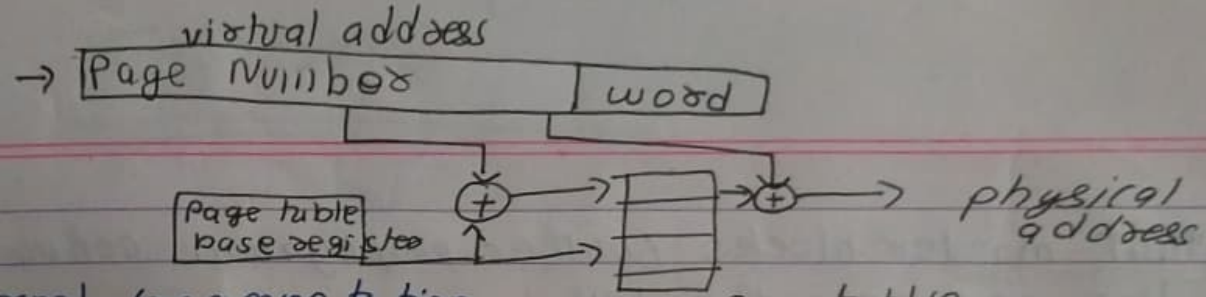
-> Paging is a memory management technique in which the process is divided into fixed size frames. Pages are stored in an available frames and a page table is used to keep track of their locations.

i Frames and Pages:
- main memory is divided into small fixed size block called frame
- The process/program is divided into block of same size called pages.

ii mapping:
- Pages of a process can be loaded into any avialable memory frame
- They donot need to be contiguous so memory is used efficiently
- The page table keeps a record ~~of~~ of which page is in which frame

iii Address translation
- when CPU accesses memory it generates a logical address (page number + offset)
- The OS uses the page table to convert it into a physical address (actual frame location)

GauravPujar



Advantages:

- solves external fragmentation
- makes memory allocation efficient
- Easy to share and manage memory between processors.

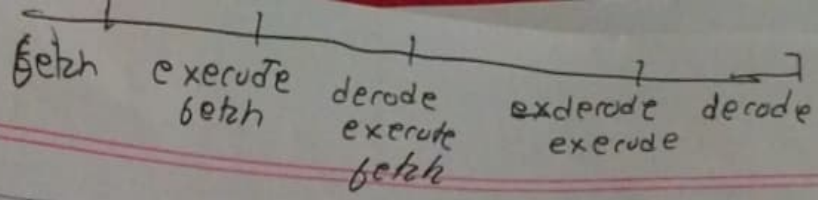
device

Q] Explain pipeline Hazards

- In CPU pipelining allows multiple instructions to be processed at the same time, each at a different stage (like fetch decode, execute)
- A pipeline hazard happens when something prevents the next instruction in the pipeline from executing in its proper clock style. In simple word, "something gets in the way and causes a delay in the pipeline.

Type of pipeline Hazards : structured Hazards
- occur when hardware resources are insufficient for executing multiple instruction at the same time
- example : if the CPU has only one memory and both instructions fetch and data access need it simultaneously one instruction has to wait.
- Solution : Add more hardware resources or stall the pipeline

Gaurav E



ii Data Hazards

- occurs when an instruction depends on the result of a previous instruction that is not yet available

Type of Data Hazards

a] RAW (Read After write)

- Instruction needs a value before it is written by the previous instruction
- Example : $I_1 : A = B + C \rightarrow I_2 : D = A + E$
- I_2 has to wait for I_1 to finish

b] WAR (write after Read)

Instruction writes to a location before it is read by a previous instruction

c] WAW [write after write]

Two instruction write to the same location in wrong order

iii Control Hazards

- Occur due to branch instruction's (like if ~~structure~~ statement or loops).
- CPU may not know which instruction to fetch next until the branch is resolved
- solution: Branch predication or stalling until branch decision is made.

Q8]

Explain branch prediction technique.

Branch Prediction:

- Branch prediction is a technique used in processor with pipelining to make instruction execution faster.
- It tries to guess the result at a conditional branch (like an if else decision) before it is actually known so the processor can continue fetching instructions without waiting.
- The part of the CPU that does this is called the branch predictor
- In purpose is to keep the instruction pipeline running smoothly.

Gaurav Pujar

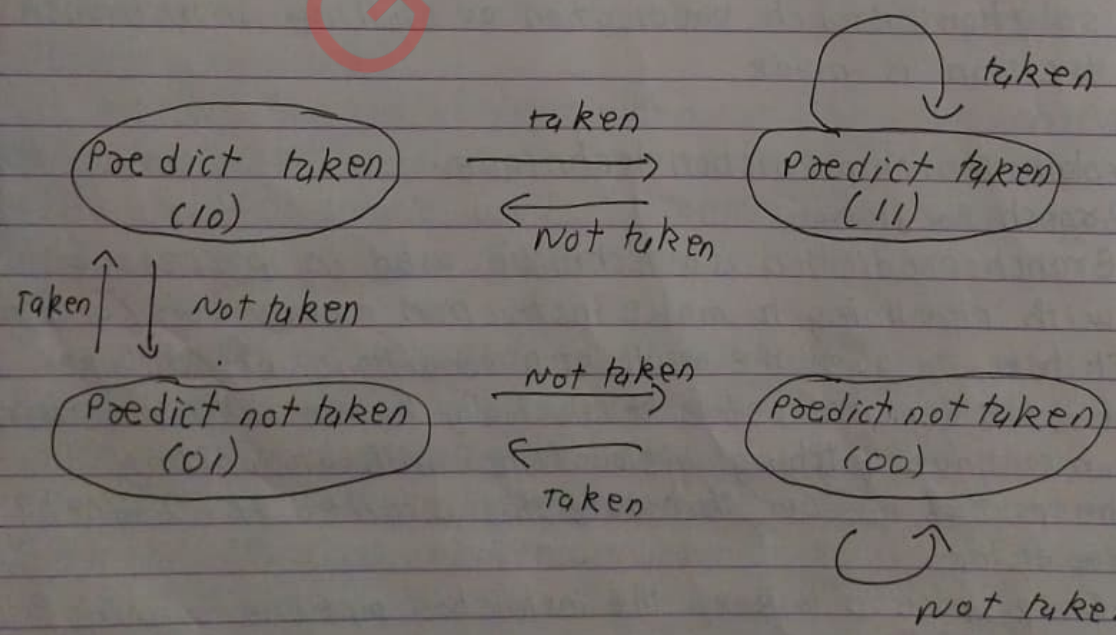
ii Static Branch Prediction

- In static prediction the decision about a branch is made at compile time and remains the same through out program execution.
- It cannot change according to program behaviour
- Two main types:
  - Hard ware - fixed prediction (decided by hardware)
  - Compiler - directed prediction (decided by compiler)

iii Dynamic Branch Prediction

- In dynamic prediction, the CPU monitors branch behaviour at run time and uses ~~fast~~ past outcomes to guess the next outcome.
- Assumption: Past behaviour helps to predict future behaviour.
- Each branch is identified by its program counter (PC) or branch history.
- Prediction involves two things:

a) outcome - whether branch will be taken or not
b) Target - the address of the next instruction.



Gaurav Pujar

~~Piple~~ Pipelinning stuges.

Instruction pipelining

Instruction pipelining is a technique of organizing the instructions for execution in such a way that the execution of the current instruction is overlapped by the execution of its subsequent instructions.

Instruction pipelining improves the performance of the processors by increasing its throughput ie number of instructions per unit time

5 stage pipelining

Fetch Instruction (FI) → Decode Instruction → Fetch operand → Execute Instruction → write operand

1	2	3	4	5	□	□	□	---
	1	2	3	4	5	□	□	--
		1	2	3	4	5	□	--
			1	2	3	4	5	--

Fetch instruction: Fetch (Read) the next instruction be be executed into instruction buffer

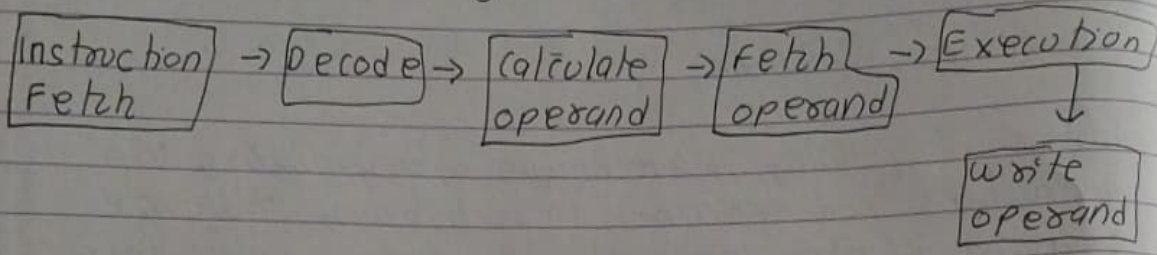
- Decode Instruction (DI): Decode the opcode

Fetch operand (FO): Read operand from memory (operand in register need not be read)

- Execute Instruction (EI): Perform the indicated operation
- write operand (WO): store result in memory

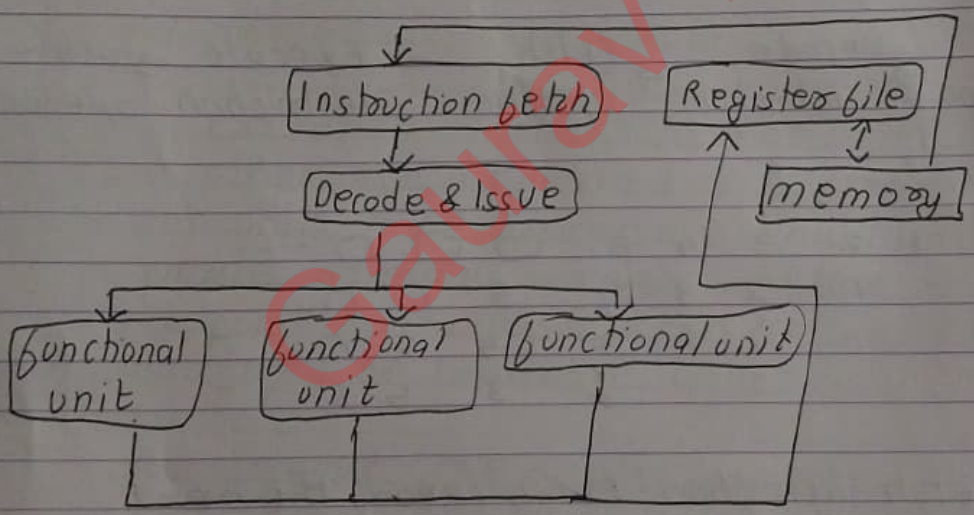
Gaurav Pujari

6 stage pipelining



1	2	3	4	5	6					
	1	2	3	4	5	6				
		1	2	3	4	5	6			
			1	2	3	4	5	6		

superscalar architecture:



superscalar technique is a CPU design that allows a processor to execute more than one instruction per clock cycle by using multiple execution units. Instead of processing instructions one after another a superscalar processor can fetch decode and execute several instructions simultaneously.

Gaurav Pujar

A superscalar processor achieves this parallelism by incorporating multiple pipelines and functional units such as arithmetic logic units (ALUs) and floating-point units (FPUs). The key steps are:

Instruction fetch: The processor fetches multiple instruction at once from the memory.

Instruction decode: These instructions are then decoded in parallel.

Dependency check: The processor's control unit ~~ato~~ analyzes the instructions to identify dependencies it checks if one instruction's result is needed as an input for another. Instructions that are independent can be executed concurrently.

Parallel execution: Instructions that don't depend on each other are dispatched to different execution units to be processed simultaneously. For example a simple integer addition and a floating point multiplication can be executed at the same time on seperate units.

Instruction Commit: The results from the completed instructions are written back to the registers or memory, maintaining the correct program order

Hazards

The primary challenge in superscalar design is dealing with hazards which are conditions that prevent parallel execution.

Gaurav Pujar

Data Hazards: occur when an instruction needs data that ~~doesn~~ hasn't been produced by a preceding instruction.

Control Hazards: Arise from conditional branch instructions as the processor may not know which brach to take until the condition is evaluated.

Structural Hazards: Happens when two instructions needs the same hardware resource at the same time.

Data Forwarding technique

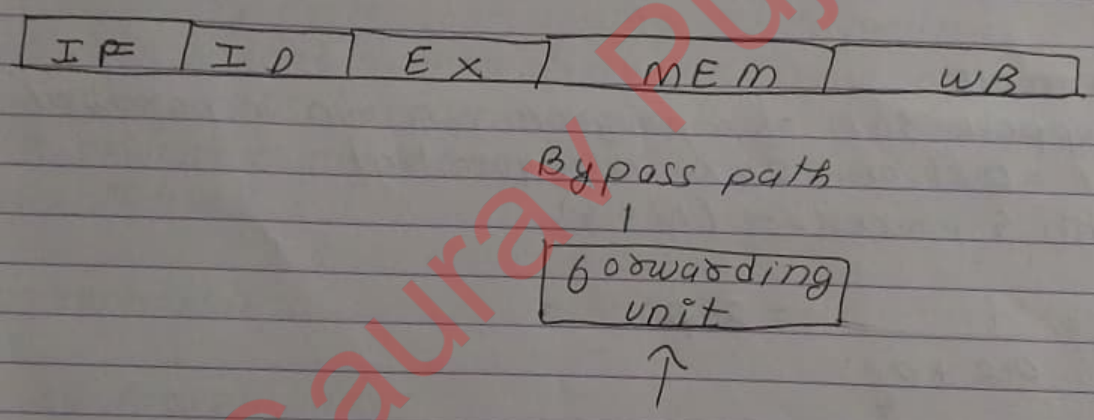
- In pipelined processors, data hazards occur when an instruction needs the result of a previous instructions that has not yet been written back to the register file. To overcome this problem and avoid pipe line stalls, the data Forwarding or data Bypassing technique is used.

- In this technique the output of the ALU or memory from one instruction is forwarded directly to the input of the next instruction that requires it, instead of waiting for it to be stored in the register. This is done through special hardware paths controlled by a forwarding unit which detects dependencies and activates the proper data path.

Gaurav Pujari

Data forwarding allows dependent instructions to execute without delay thereby reducing stalls and improving pipeline performance. However it can not remove all hazards such as load use hazards where data is still being fetched from memory

Hence data forwarding is an efficient method to mitigate data hazards and maintain smooth instructions flow in pipelined processors.



Q) write short note on Amdahl's low.

Defination:
Amdah's low is used in computer architecture to find the maximum possible speed up of a program when only part of program can be parallelized.

Formula

$$\text{Speedup} = \frac{1}{(1-P)+\frac{P}{N}}$$

Gaurav Pujar

where

P = fraction of program that can be parallelized

$(1-P)$ = fraction of program that is sequential

N = Number of processors.

Key Idea:

If a program has a portion that must run sequentially then adding more processors will only speed up the parallel part.

The sequential part limits the maximum speed up.

Example

- Suppose 80% of a program can run in parallel ($P = 0.8$) and 20% is sequential.
- with 4 processors ($N = 4$)

$$S = \frac{1}{0.2 + \frac{0.8}{4}} = 2.5$$

So even with 4 processors the program run 2.5 times faster, not 4 times

Importance:

- shows the limitation of parallel ~~processing~~ computing
- Helps computer architecture decide whether adding more processors is useful
- Proves that beyond a point increasing processors gives diminishing returns.

Gaurav Pujar

The 80386 DX processor operates in three modes

1] Real mode

Purpose : To provide compatibility with earlier processors like the Intel 8086

80386 real mode is the default operating mode of the Intel 80386 (and later x86 processors) where the processor functions like a legacy 8086 CPU offering 1MB of addressable memory without advanced features like memory protection, paging. This mode is used for initial system startup and setup before the processor transitions to more advanced modes like protected mode to utilize its full 32 bits capabilities.

characteristics

legacy compatibility:
The 80386 in real mode behaves like 8086, allowing software written for older processors to run without modification.

16 bit operations:
All the operations in real mode are limited to 16 bits including registers and instructions similar to the 8086

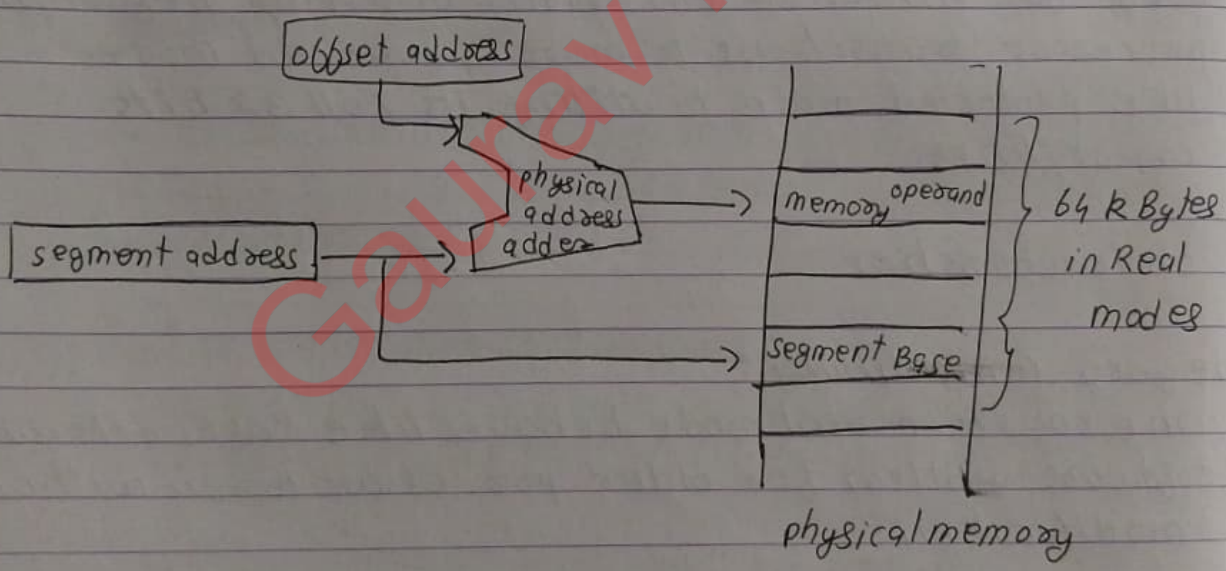
1MB address space
The processor can only access 1MB of physical memory using 20 address lines (A0 - A19)

Gaurav Pujar

segment size: The maximum segment size in real mode is 64 KB due to 16 bit offset.

Disabled Advanced features:
Key 80386 features such as protected memory paging, virtual memory and multiasking are unavailable in real mode.

Initial Power up state
After a system reset or power up the processor defaults to real mode to initialize and set up the system for its more advanced modes.



Gaurav Pujar

Protected mode

Purpose : To enable the full 32 bit capabilities of 80386 including advanced memory management and multitasking

The 80386's protected mode is its most advanced operating mode unlocking its full 32 bit architecture and feature like memory protection, multitasking, segmentation and paging. It provides access to 4GB of physical memory and 64TB of virtual memory by using selector in segment registers to access segment descriptor which contain base address and limits. This mode allows operating systems to isolate processes; ~~manang~~ manage memory efficiently multitasking applications securely.

Key features of protected mode

Segmentation : Divides memory into logical units called segments, allowing the 80386 to map virtual addresses to physical addresses.

Paging : Further divides memory into fixed size pages (4KB) for efficient memory management and to enable virtual memory where parts of memory can be swapped to and form disk

Memory protection : Provides mechanisms for the operating system to protect memory and prevent applications from interfering with each other or with the system itself

Multitasking support : Facilitates multitasking by allowing multiple programs to run concurrently with memory protection and task switching capabilities.

Large Address space : grant access to a 32 bit linear address space of 4GB of physical memory.

Gaurav Pujar

Multiple Privilege levels:
Implements a hierarchical protection model with four privilege level to control access to system resources and prevent applications from performing privillaged operations.

How protected mode works:

Transition: The 80386 starts in Real mode and then transitions to protected mode by setting the protection enable (PE) bits in the control register CR0

Segmentation: segment registers contain selectors which are used to access segment's descriptions in memory These descriptors store the segment base address, limit and access rights.

Address translation: The 80386 uses the segment base address and the offset from the segment register to form a linear address.

Paging (opt): if enabled the paging unit translates their linear address into physical address providing the final memory location.

Gaurav Pujar

Virtual 8086 mode

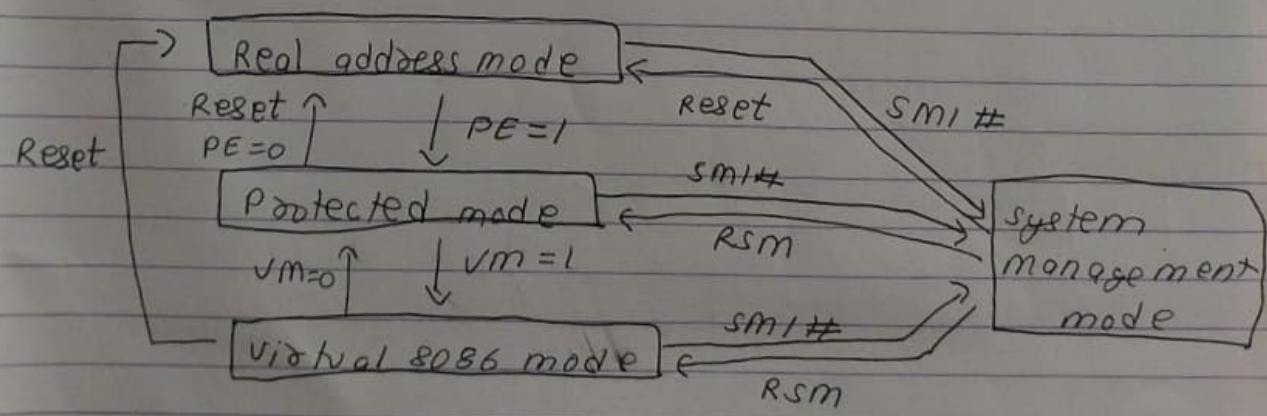
Virtual 8086 mode (often written as V86 mode) is a special operating mode of the Intel x86 processor (introduced with the 80386 CPU) that allows the execution of real mode (8086) applications inside a protected mode environment.

Key features

1) Real mode compatibility
- Executes 8086 instructions as if the CPU were in real mode
- Uses 16 bit register, 20 bit addressing (1MB address space)

2) Run inside protected mode
- The processor still operates in protected mode
- Each V86 task has segmented memory addressing but the OS can enforce protection.

3 multiple V86 tasks
An operating system can run multiple DOS applications at once each in its own virtual 8086 mode environment.



Gaurav Pujar